

ML Project Phase 2

1. **Project Title:** Vehicle Classification System

2. **Background and Motivation:**

Vehicle classification is an important computer vision task with applications in intelligent transportation systems, autonomous vehicles, traffic monitoring, surveillance, and smart city development. Manual identification of vehicles from images is often inefficient and prone to errors when processing large volumes of visual data. Recent developments in Deep Learning, particularly Convolutional Neural Networks (CNNs), have demonstrated remarkable success in image classification tasks by automatically learning discriminative visual features from images. This project aims to develop a CNN-based vehicle classification system capable of identifying seven vehicle categories, namely Cars, Motorcycles, Bikes, Trains, Ships, Planes, and Auto Rickshaws. The outcomes of this research may contribute to the development of intelligent vehicle recognition systems for transportation, surveillance, and smart city applications.

3. **Problem Statement:**

Manual vehicle classification from images is time-consuming and prone to errors. Therefore, an automated deep learning-based system is required to accurately classify different types of vehicles from images.

4. **Selected dataset and dataset link:**

Vehicle Image Classification

<https://www.kaggle.com/datasets/mohamedmaher5/vehicle-classification>

5. **Dataset description:**

Number of images: 5,588 images

Number of classes: 7

Image type: RGB Images (.jpg, .png, .PNG)

6. Research questions:

RQ1: How effectively can a custom Convolutional Neural Network (CNN) classify vehicle images into seven vehicle categories?

RQ2: Does transfer learning using MobileNetV2 improve vehicle classification performance compared to a custom CNN model?

RQ3: How do CNN, MobileNetV2, and EfficientNetB0 compare in terms of accuracy, precision, recall, and F1-score for vehicle image classification?

RQ4: Which vehicle categories are most frequently misclassified, and what insights can be obtained from confusion matrix and error analysis?

RQ5: Can SHAP (SHapley Additive exPlanations) effectively explain the prediction decisions made by the best-performing vehicle classification model?

7. Expected results:

The proposed vehicle classification system is expected to accurately classify vehicle images into seven categories: Cars, Motorcycles, Cycles, Trains, Ships, Planes, and Auto Rickshaws. The custom CNN model is expected to achieve satisfactory performance, while transfer learning models, particularly MobileNetV2, are expected to provide higher classification accuracy and better generalization due to pretrained feature extraction. SHAP analysis is expected to improve model interpretability by highlighting image regions that contribute most to classification decisions.

8. Proposed Methodology:

- **Dataset Collection and Loading:** Load the Vehicle Classification dataset containing 5,588 images across seven vehicle classes.
- **Data Preprocessing:** Resize all images to 224×224 pixels.
- Normalize pixel values to improve model convergence.
- Split the dataset into training, validation, and testing sets.
- **Data Augmentation:** Apply random flipping, rotation, zooming, and contrast adjustments to increase dataset diversity and reduce overfitting.
- **Model Development:** Develop a custom CNN architecture for vehicle classification.
- Implement transfer learning models for comparison.

- **Model Training:** Train models using the training dataset.
- Use Early Stopping and ReduceLROnPlateau to prevent overfitting and optimize learning.
- **Model Evaluation:** Evaluate models using accuracy, precision, recall, F1-score, ROC curves, confusion matrices, and classification reports.
- **Explainability Analysis:** Apply SHAP to interpret model predictions and identify important image features.
- **Model Comparison:** Compare the performance of all models and identify the best-performing approach.

9. CNN Model Design:

The custom CNN architecture consists of multiple convolutional and max-pooling layers followed by fully connected layers for classification.

Layer	Configuration
Input Layer	$224 \times 224 \times 3$
Conv2D + ReLU	32 Filters
MaxPooling2D	2×2
Conv2D + ReLU	64 Filters
MaxPooling2D	2×2
Conv2D + ReLU	128 Filters
MaxPooling2D	2×2
Conv2D + ReLU	256 Filters
MaxPooling2D	2×2
Flatten	Feature Extraction
Dense + ReLU	256 Units

Layer	Configuration
Dropout	0.5
Output Layer	7-Class Softmax

10. Transfer Learning Models

- MobileNetV2**
Pretrained on ImageNet.
Lightweight architecture with reduced computational complexity.
Used as a feature extractor with a custom classification head.
- EfficientNetB0**
Pretrained on ImageNet.
Uses compound scaling to balance network depth, width, and resolution.
Evaluated for comparison with MobileNetV2 and the custom CNN.

11. Evaluation Metrics

The following metrics are used to assess model performance:

Metric	Purpose
Accuracy	Overall classification performance
Precision	Correct positive predictions
Recall	Ability to identify actual class instances
F1-Score	Balance between precision and recall
Confusion Matrix	Visualization of classification performance
ROC Curve	Class-wise discrimination ability
Classification Report	Detailed class-wise performance evaluation
SHAP Analysis	Model interpretability and feature importance

12. Expected Figures

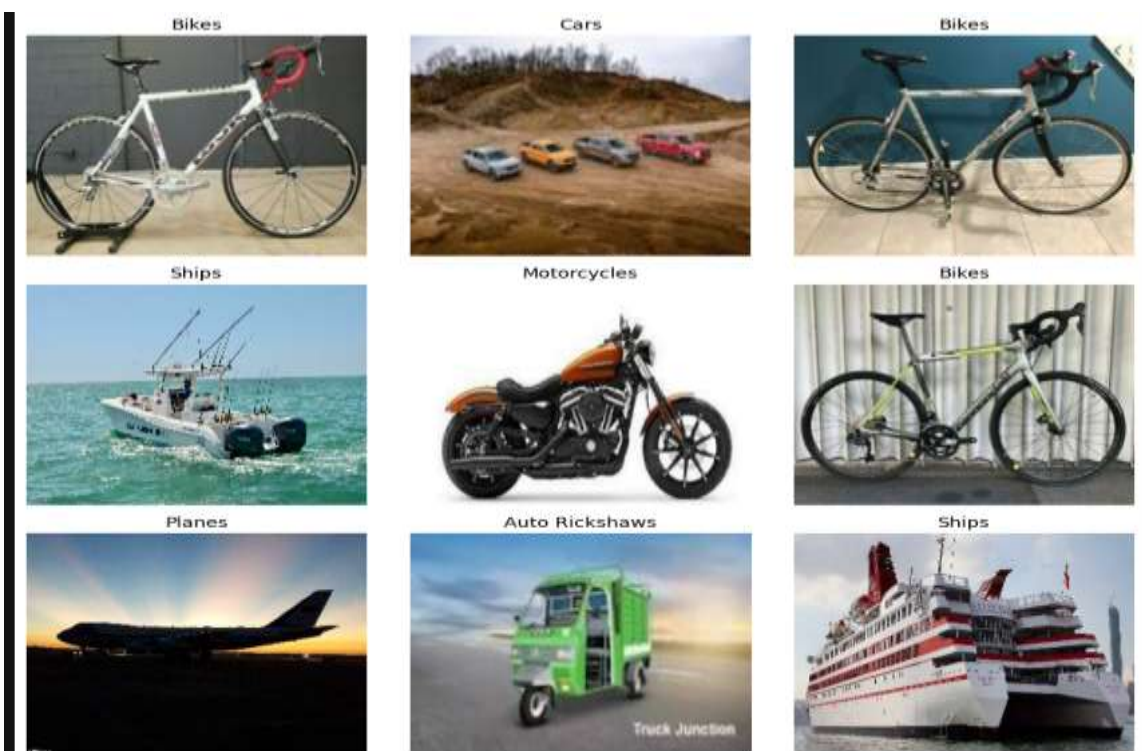
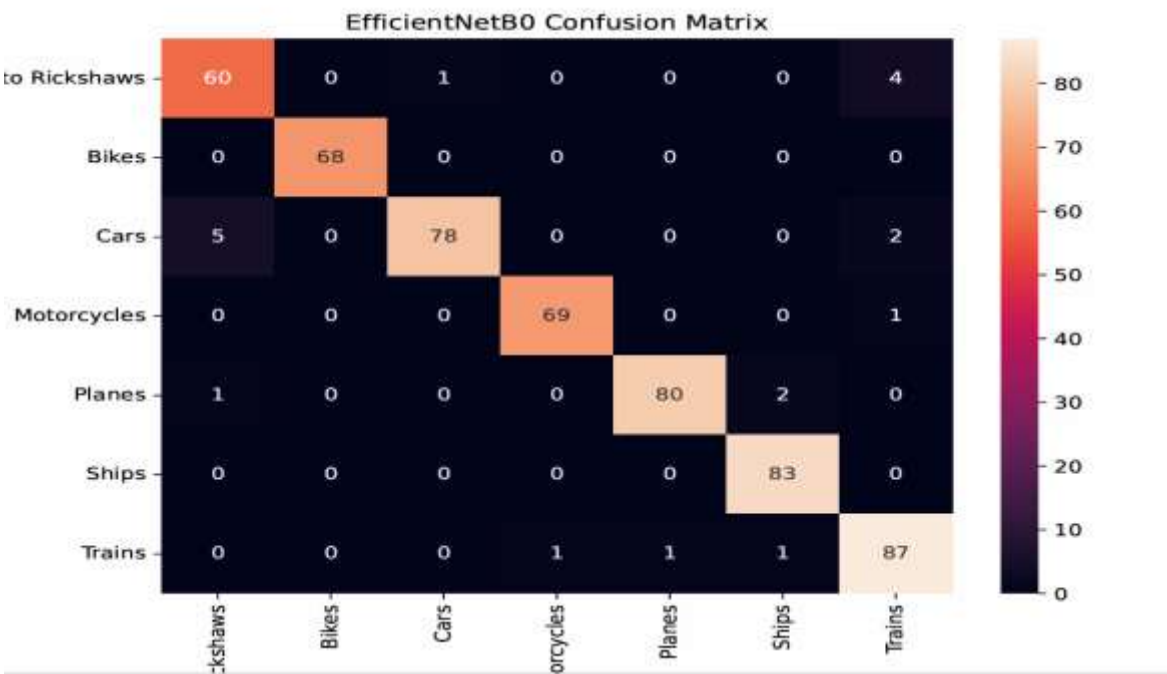


Fig 1: Sample Vehicle Images from the Dataset



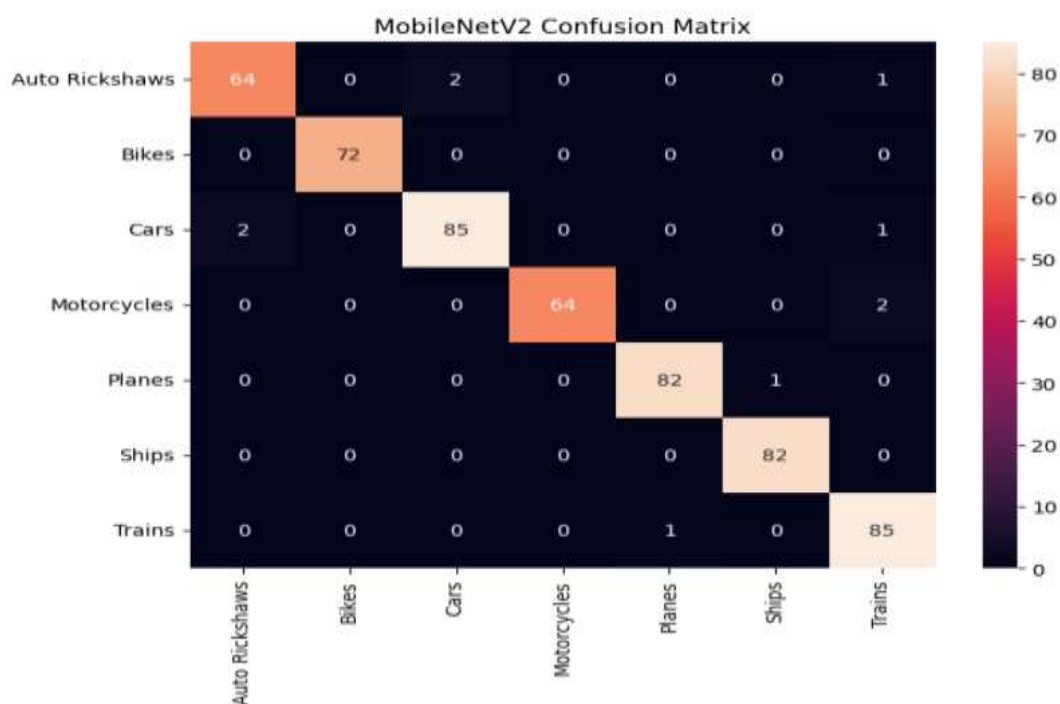
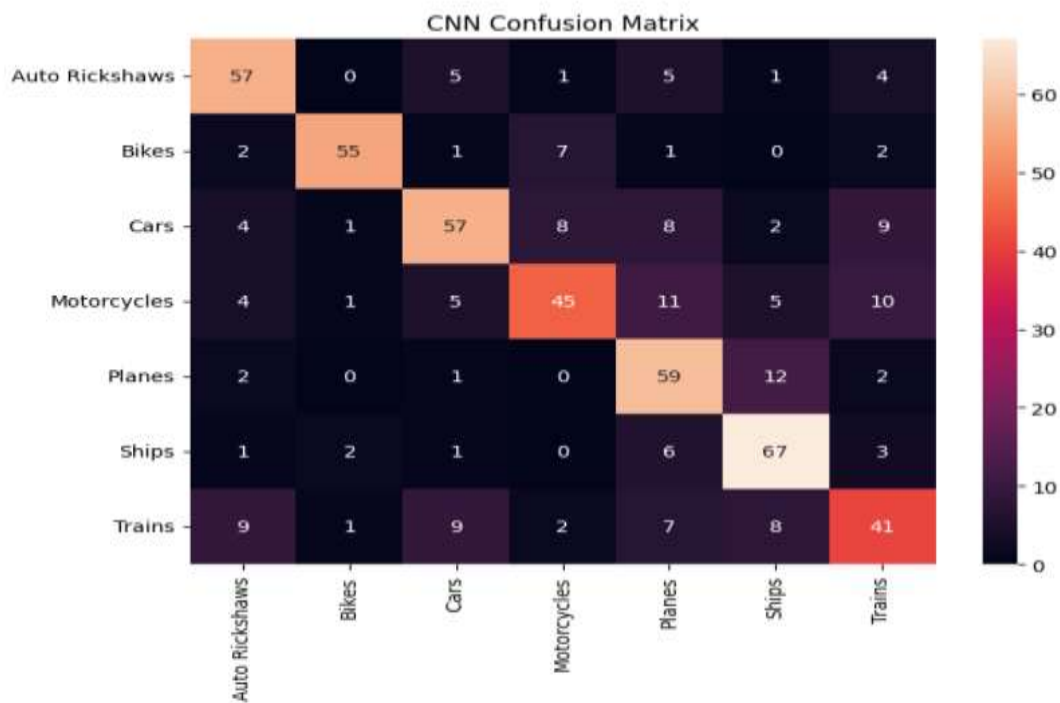


Fig 2: Confusion Matrix (EfficientNet B0, CNN, MobileNet V2)



Fig 3: Shap Diagram

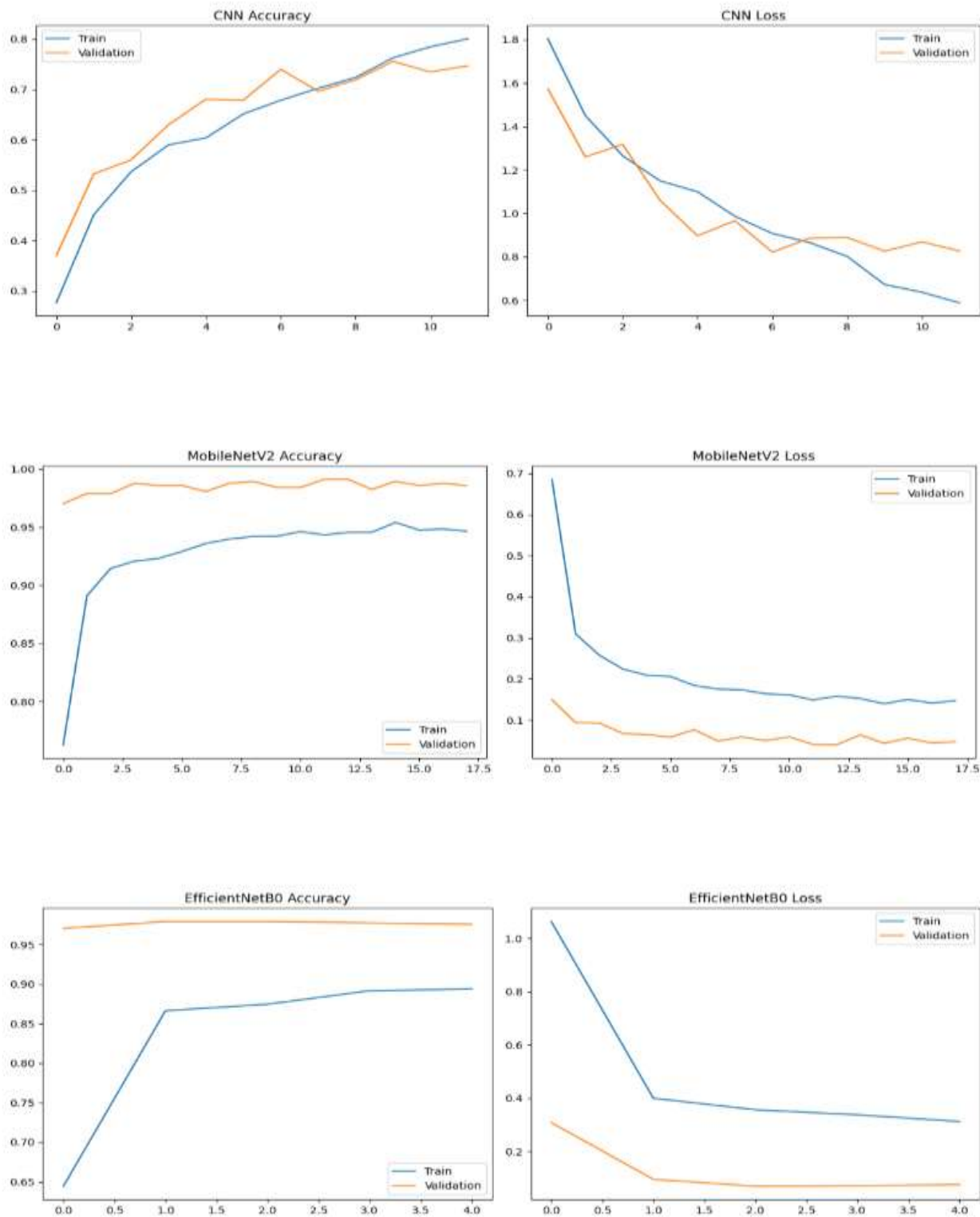


Fig 4: Models Accuracy and Loss



Fig 5: True vs Prediction

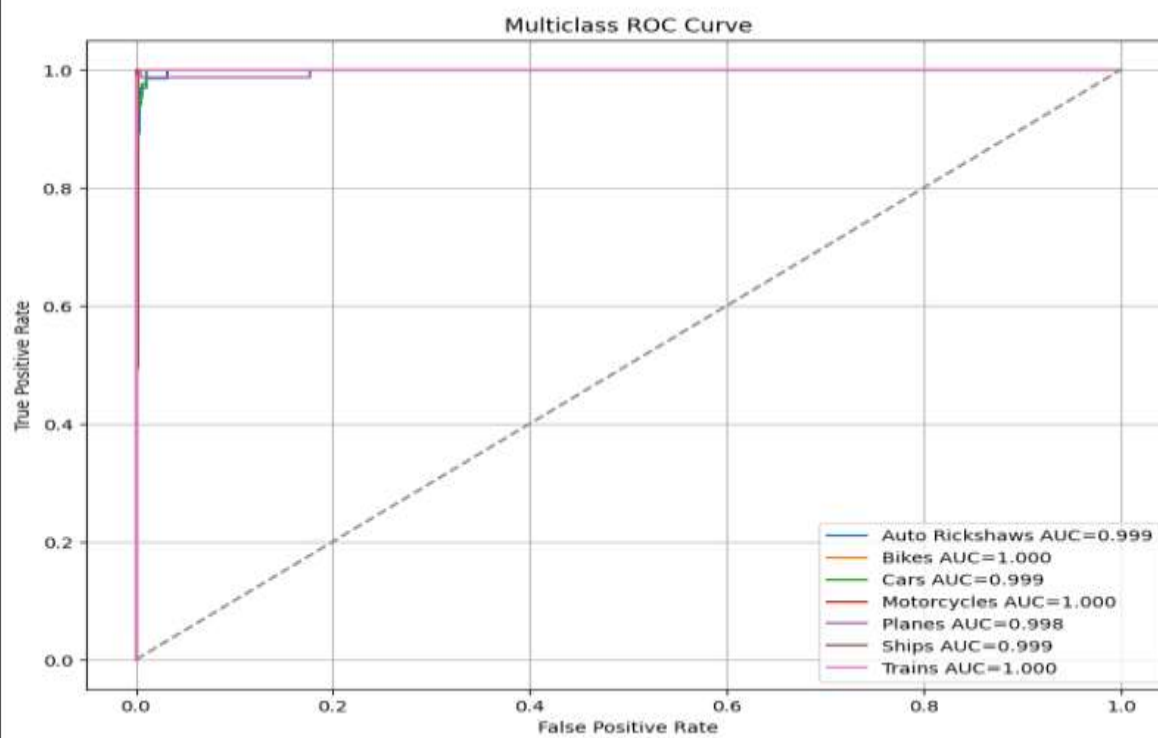


Fig 6: ROC Curve

13. Expected Tables

	Model	Accuracy	F1
1	MobileNetV2	0.981618	0.981613
2	EfficientNetB0	0.965074	0.965086
0	CNN	0.700368	0.698461

Table 1: Model Comparison(Classification Report)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 224, 224, 3)	0
rescaling (Rescaling)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout_1 (Dropout)	(None, 1280)	0
dense_2 (Dense)	(None, 7)	8,967

Total params: 2,284,887 (8.72 MB)
Trainable params: 8,967 (35.03 KB)
Non-trainable params: 2,257,984 (8.61 MB)
Optimizer params: 17,936 (70.07 KB)

Table 2: Summary Table